

1. Exercise 8.8.1: Let p be a prime and let a be an integer where p does not divide a . Let $h(x) = a^x \pmod{p}$.

Explain why $h(x)$ is not a good cryptographic hash function.

Fermat's Little Theorem

Let p be a prime which does not divide the integer a , then $a^{p-1} = 1 \pmod{p}$.

Proof.

Start by listing the first $p-1$ positive multiples of a :

$a, 2a, 3a, \dots, (p-1)a$

If ra and sa are the same modulo p , then we have $r = s \pmod{p}$, so all $p-1$ multiples of a above are distinct and nonzero; that is, they must be congruent to $1, 2, 3, \dots, p-1$.

Multiply all these congruencies together and we find $a \cdot 2a \cdot 3a \cdot \dots \cdot (p-1)a = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-1) \pmod{p}$ or better, $a^{(p-1)}(p-1)! = (p-1)! \pmod{p}$.

Above, all multiplication groups modulo a prime are cyclic. This means that for the chosen prime p , there exists at least one generator g such that $g^k = 1 \pmod{p}$ for some integer k and it is not difficult to find such a generator. If the base a happens to be a generator (and the percentage of group members that are generators is higher than intuition might suggest), then you can easily construct two messages, x and $x+nk$ (n being another positive integer), that will produce the same "hash." This would violate a basic property required of cryptographic; the hash must be collision free.

In the above hash equation, $h(x)$ is computed as $a^x \pmod{p}$. If a suitable x can be practically computed from a given hash h , we have found a message that yields the given hash. This type of computation, the inverse of modular exponentiation, is a discrete logarithm. If the discrete logarithm can feasibly be computed, and a file found corresponds to a given hash, then the hash has poor collision free.

Now, using Fermat's Little Theorem.

$a^{(p-1)} \equiv 1 \pmod{p}$, where $p \nmid a$

$h(x_1) \equiv a^{x_1} \equiv a^{x_1} a^{(p-1)} \equiv a^{(x_1 + (p-1))} \pmod{p}$

If $x_2 = x_1 + (p-1)$, where $h(x_1) = h(x_2)$ but $x_1 \neq x_2$.

Example:

$h(x) = a^x \pmod{p} \rightarrow \text{modp}(\text{Power}(5,3),7);$

Assume $p=7$, $a=5$. And find different values of x .

Since we get the same value for $x=3$, $x=9$, $x=15$;

$h(x)$ is not a good hash function

2. Exercise 8.8.2: Let $n=pq$ be the product of two distinct large primes and let $h(x) = x^2 \pmod{n}$.

Why is h pre-image resistant (when x^2 is not a perfect square in the integers)?

To prove that h is pre-image resistant we will proceed to decrypt h .

Therefore $x = (x^2)^d = x \pmod{n}$, from the following;

$$m^{k \cdot \phi(n) + 1} = m \pmod{n}$$

$$m^{ed} = m \pmod{n},$$

We have that $ed = 1 \pmod{\phi(n)} \rightarrow 2d = 1 \pmod{\phi(n)}$

$d = \frac{1 \pmod{\phi(n)}}{e}$, so in order for us to get d we need to get the inverse of e .

As we see, $e = 2$ and 2 does not have an inverse which therefore makes impossible for us to find an exponent d to decrypt the message. This makes the hash one way only.

On the other hand, $\gcd(2, \phi(n)) \neq 1$ as $\phi(n) = (p-1)(q-1)$, which is the product of even integers. This makes finding x computationally infeasible.

Why is h not collision resistant?

For a given n a list of the quadratic residues modulo n may be obtained by simply squaring the numbers $0, 1, \dots, n-1$. Because $x^2 \equiv (n-x)^2 \pmod{n}$, the list of squares modulo n is symmetrical around $\frac{n}{2}$. Thus, the number of quadratic residues modulo n cannot exceed $\frac{n}{2} + 1$ (n even) or $\frac{(n+1)}{2}$ (n odd).

Modulo an odd prime number p there are $\frac{(p+1)}{2}$ residues (including 0) and $\frac{(p-1)}{2}$ non-residues. Every congruence class except zero modulo p has a multiplicative inverse. The first supplement to the law of quadratic reciprocity is that if $p \equiv 1 \pmod{4}$ then -1 is a quadratic residue modulo p , and if $p \equiv 3 \pmod{4}$ then -1 is a non-residue modulo p .

This implies the following:

If $p \equiv 1 \pmod{4}$ the negative of a residue modulo p is a residue

If $p \equiv 3 \pmod{4}$ the negative of a residue modulo p is a non-residue

Where $p_1 \neq p_2$, but $p_1^2 = p_2^2$

In a more genetic way, we are saying that the square of a number x and the square of a number $p-x$ are the same module p , which can be proved as follows:

$$x^2 \pmod{p} = (n-x)^2 \pmod{p} \text{ but } x \neq n-x.$$

Let $x_1 = x$, and $x_2 = n-x$. Then, we have that two messages contain the same hash since $x_1^2 = x_2^2$ but $x_1 \neq x_2$. Therefore, this hash h is not collision free.

3. Exercise 8.8.3. Suppose a message m is divided into blocks of 160 bits. $m = m_1 m_2 \dots m_l$. Let $h(m) = m_1 \text{ XOR } m_2 \text{ XOR } \dots \text{ XOR } m_l$. Which of these three properties does h satisfy?

1. Fast to compute
2. Pre-image resistant
3. Collision resistant

Explain your answer.

1) (+) is a simple operation, and therefore, $h(x)$ can be easily calculated.

2) $h(x)$ does not satisfy the pre-image resistant because of the following reasons:

Given $h(x)$, we can construct $m_1(+)m_2(+) \dots (+)m_l$ in arbitrary manner. Since we know $h(m)$, we have a 160-bit string of 0s and 1s. For each 0 in $h(m)$, we can assign an even number of 1s (or all 0s) to that position in the l blocks.

3) $h(x)$ does not satisfy the collision-free because of the following reasons:

We are only looking for $m_1 \neq m_2$ and $h(m_1) = h(m_2)$. We do not want to base this on a given $h(m)$. So, we can arbitrarily construct $m_1(+)m_2(+) \dots (+)m_l$ for some number of blocks l . We could change an even number of 1s from a given position to 0, from 0 to 1, etc. and this changes the message but not the hash value.

4) **Computer Problem 8.9.2.** What is the probability of at least two students in a class of 200 having the same last four digits in their social security numbers? Compute the exact answer and compare this to the approximation given by formula (8.1) (page 230): Probability of a match with r people, n possibilities = $1 - e^{-r^2/(2n)}$

Briefly explain how you computed your answers, and attach your code, if any.

To compute the above problem I proceeded to do code the following formula:

$$P(e) = (1 - i/m) * \dots (1 - in/m)$$

Here, i is the number of students starting from 0 to 199 where $in = 199$, and m is the greatest last four digits of a social security, in this case 9999.

To obtain the result in a quickly manner I built a simple that those the above calculation, this code is shown below:

Code:

```
public class Probability {

    public static void main(String[] args){

        int n = 200, m = 9999;

        probability(n, m);

    }

    private static void probability(int n, int m) {

        double i = 0;
        double pe = (1 - (i/m));
        ++i;
        System.out.println("Decimal Estimate" + "\t" + "Percentage Estimate");
        System.out.println(pe + "\t\t" + pe*100+"%");

        while(i++ != n){

            pe *= (1 - (i/m));
            System.out.println(pe + "\t" + pe*100+" %");

        }

    }

}
```

Output:

Decimal Estimate	Percentage Estimate
1.0	100.0%
0.9997999799979999	99.97999799979999 %
after ...	
0.21621370064300321	21.621370064300322 %
0.21242958246993335	21.242958246993336 %
0.20869044790500604	20.869044790500606 %
0.20499625755805276	20.499625755805276 %
0.20134695924368798	20.1346959243688 %
0.1977424882261242	19.77424882261242 %
0.19418276746597796	19.418276746597797 %
0.19066770786888404	19.066770786888405 %
0.18719720853573704	18.719720853573705 %
0.1837711570143809	18.37711570143809 %
0.18038942955257012	18.038942955257014 %
0.1770518913520275	17.70518913520275 %
0.17375839682342692	17.375839682342693 %
0.1705087898421307	17.05087898421307 %
0.1673029040045149	16.73029040045149 %
0.1641405628847176	16.41405628847176 %
0.16102158029164865	16.102158029164865 %
0.1579457605261016	15.794576052610159 %
0.15491289863781163	15.491289863781162 %
0.15192278068230633	15.192278068230634 %
0.1489751839773991	14.897518397739908 %
0.14606987735917798	14.606987735917798 %
0.14320662143734592	14.320662143734591 %
0.14038516884977145	14.038516884977145 %
0.1376052645161126	13.76052645161126 %
0.1348666458903794	13.48666458903794 %
0.132169043212304	13.2169043212304 %

Resultl Decimal Estimate	Result Percentage Estimate
0.8678309567876961	86.78309567876961 %

Out of 200 students, there is 86.783% of probability that two students share the same last 4 digits of their social security number.

$$1 - e^{-200 \cdot 200 / (2 \cdot 9999)} = 0.86466 \approx 86.783$$